# LucaAgent User E2E Testing Guide

## Complete End-to-End Testing Coverage for All User Roles

**Version:** 1.0.0
**Last Updated:** January 2, 2026
**Scope:** All user roles, subscription tiers, and feature interactions

---

## 🔑 Quick Start: Test Accounts

**Password for ALL test accounts:** `TestPassword123!`

```
# Seed all test accounts
npx tsx scripts/seed-test-users.ts
```

| Subscription | Test Account | What You Can Test |
|---|---|---|
| **Super Admin** | `superadmin@lucatest.com` | System monitoring, deployments, all admin features |
| **Admin** | `admin@lucatest.com` | User management, training data, analytics |
| **Enterprise** | `enterprise.owner@lucatest.com` | SSO, multi-user teams, custom AI, unlimited features |
| **Professional** | `professional.cpa@lucatest.com` | API access, forensic intelligence, white-label |
| **Plus** | `plus.business@lucatest.com` | 3K queries, 5 profiles, scenario simulator |
| **Free** | `free.home@lucatest.com` | 500 queries, 10 docs, 1 profile, basic features |
| **New User** | `free.newuser@lucatest.com` | Onboarding flow (no profile created) |
| **Quota Limit** | `quota.exhausted@lucatest.com` | Upgrade prompts, limit errors |
| **MFA Enabled** | `mfa.enabled@lucatest.com` | 2FA login flow |
| **Locked** | `locked.account@lucatest.com` | Account lockout recovery |

> ⚠️ ***Add `superadmin@lucatest.com` to `ADMIN_EMAIL_WHITELIST` env variable!***

---

## Table of Contents


1. Role Definitions
2. Test Environment Setup
3. Authentication Test Suite
4. Role-Based Access Control Tests
5. Subscription Tier Tests
6. Feature Tests by Role
7. Profile & Member Tests
8. Cross-Role Interaction Tests

9. Security Tests
10. Edge Cases & Error Handling
11. Performance Tests
12. Test Data Management

---

# 1. Role Definitions

## 1.1 System Roles

| Role | Identifier | Description |
|---|---|---|
| **Super Admin** | Email in `ADMIN_EMAIL_WHITELIST` | Highest privilege - system control |
| **Admin** | `users.isAdmin = true` | Platform administration |
| **Regular User** | `users.isAdmin = false` | Standard authenticated user |
| **Guest** | No session | Unauthenticated visitor |

## 1.2 Profile Member Roles

| Role | Permissions |
|---|---|
| **Owner** | Full control, can delete profile, manage all members |
| **Admin** | Manage members, edit profile settings |
| **Member** | Basic read/write access to profile data |
| **Viewer** | Read-only access |

## 1.3 Subscription Tiers

| Tier | Query Limit | Documents | Profiles | Features |
|---|---|---|---|---|
| **Free** | 500/month | 10/month | 1 | Basic analysis |
| **Plus** | 3,000/month | Unlimited | 5 | Scenario Simulator, Deliverables |
| **Professional** | Unlimited | Unlimited | Unlimited | API, Forensic, White-label |
| **Enterprise** | Unlimited | Unlimited | Unlimited | SSO, Multi-user, Custom AI |

---

# 2. Test Environment Setup

## 2.1 Pre-Seeded Test User Accounts

Run the seed script to create all test users automatically:

```
npx tsx scripts/seed-test-users.ts
```

🔑 **Password for ALL test accounts:** `TestPassword123!`

**Super Admin & Admin Accounts**

| Email | Role | Tier | Use Case |
|---|---|---|---|
| `superadmin@lucatest.com` | Super Admin | Enterprise | System monitoring, deployments, maintenance mode |
| `admin@lucatest.com` | Admin | Professional | User management, training data, coupons, analytics |

> ⚠️ ***Important:*** *Add* `superadmin@lucatest.com` *to the* `ADMIN_EMAIL_WHITELIST` *environment variable!*

**Enterprise Tier Accounts (Corporate)**

| Email | Profile Role | Use Case |
|---|---|---|
| `enterprise.owner@lucatest.com` | Owner | Corporate owner - SSO, multi-user, custom AI training |
| `enterprise.cfo@lucatest.com` | Admin Member | CFO - team admin permissions |
| `enterprise.accountant@lucatest.com` | Member | Staff accountant - basic member permissions |

**Team Structure:**

```
Acme Corporation (Business Profile)
├── enterprise.owner@lucatest.com (Owner) - Full control
├── enterprise.cfo@lucatest.com (Admin) - Manage members, edit settings
└── enterprise.accountant@lucatest.com (Member) - Add data, view only
```

**Professional Tier Accounts**

| Email | Profile Type | Use Case |
|---|---|---|
| `professional.cpa@lucatest.com` | Business | CPA practice - API access, forensic intelligence, white-label |
| `professional.consultant@lucatest.com` | Business | Tax consultant - unlimited queries/profiles |

**Plus Tier Accounts**

| Email | Profile Type | Use Case |
|---|---|---|
| `plus.business@lucatest.com` | Business | Small business owner - 3K queries, 5 profiles, scenarios |
| `plus.family@lucatest.com` | Family (Owner) | Family finances - multi-member household |
| `plus.family.spouse@lucatest.com` | Family (Admin) | Spouse with admin permissions |

| | | |
|---|---|---|
| `plus.family.child@lucatest.com` | Family (Viewer) | College student - view only access |
| `plus.freelancer@lucatest.com` | Personal | Freelancer/sole proprietor |

**Family Structure:**

```
Johnson Family Finances (Family Profile)
├── plus.family@lucatest.com (Owner) - Full control
├── plus.family.spouse@lucatest.com (Admin) - Manage finances, invite
└── plus.family.child@lucatest.com (Viewer) - View only
```

**Free Tier Accounts (Home/Personal)**

| Email | Profile | Use Case |
|---|---|---|
| `free.home@lucatest.com` | Personal | Standard home user - 500 queries, 10 docs, 1 profile |
| `free.student@lucatest.com` | Personal | Student - simple tax situations |
| `free.newuser@lucatest.com` | **None** | Fresh account - tests onboarding flow |

**Edge Case Accounts**

| Email | Condition | Test Purpose |
|---|---|---|
| `quota.exhausted@lucatest.com` | At quota limit | Upgrade prompts, limit error messages |
| `mfa.enabled@lucatest.com` | MFA enabled | 2FA login flow, TOTP verification |
| `locked.account@lucatest.com` | Account locked | Lockout recovery, unlock flow |

## 2.2 Feature Access by Subscription Tier

Use this matrix to know what each test account can access:

| Feature | Free | Plus | Professional | Enterprise |
|---|---|---|---|---|
| **Chat Queries** | 500/mo | 3,000/mo | Unlimited | Unlimited |
| **Document Uploads** | 10/mo | Unlimited | Unlimited | Unlimited |
| **Profiles** | 1 | 5 | Unlimited | Unlimited |
| **Export: TXT/CSV** | ✅ | ✅ | ✅ | ✅ |
| **Export: PDF/DOCX/XLSX** | ❌ | ✅ | ✅ | ✅ |
| **Scenario Simulator** | ❌ | 10/mo | Unlimited | Unlimited |
| **Deliverable Composer** | ❌ | 10/mo | Unlimited | Unlimited |
| **Deep Research Mode** | ❌ | ✅ | ✅ | ✅ |
| **Forensic Intelligence** | ❌ | ❌ | ✅ | ✅ |

| | | | | |
|---|---|---|---|---|
| **API Access** | ❌ | ❌ | ✅ | ✅ |
| **White-label Reports** | ❌ | ❌ | ✅ | ✅ |
| **SSO/SAML** | ❌ | ❌ | ❌ | ✅ |
| **Multi-user (6+)** | ❌ | ❌ | ❌ | ✅ |
| **Custom AI Training** | ❌ | ❌ | ❌ | ✅ |
| **SLA Guarantee** | ❌ | ❌ | ❌ | 99.9% |

### 2.3 Profile Member Permissions

| Action | Owner | Admin | Member | Viewer |
|---|---|---|---|---|
| View profile data | ✅ | ✅ | ✅ | ✅ |
| Edit profile settings | ✅ | ✅ | ❌ | ❌ |
| Add documents/data | ✅ | ✅ | ✅ | ❌ |
| Invite new members | ✅ | ✅ | ❌ | ❌ |
| Change member roles | ✅ | ❌ | ❌ | ❌ |
| Remove members | ✅ | ✅* | ❌ | ❌ |
| Delete profile | ✅ | ❌ | ❌ | ❌ |
| Transfer ownership | ✅ | ❌ | ❌ | ❌ |

*Admin can remove non-admin members only

### 2.4 Which Account to Use for Each Test

| Test Scenario | Recommended Account | Why |
|---|---|---|
| Basic chat functionality | `free.home@lucatest.com` | Tests core features without premium |
| Quota exceeded errors | `quota.exhausted@lucatest.com` | Already at limit |
| Upgrade flow prompts | `free.home@lucatest.com` | Triggers upgrade CTAs |
| PDF/DOCX export | `plus.business@lucatest.com` | Has export access |
| Scenario simulator | `plus.freelancer@lucatest.com` | Has simulator access |
| Forensic analysis | `professional.cpa@lucatest.com` | Has forensic access |
| API key management | `professional.consultant@lucatest.com` | Has API access |
| SSO configuration | `enterprise.owner@lucatest.com` | Has SSO access |
| Team member permissions | `enterprise.accountant@lucatest.com` | Limited team role |

| | | |
|---|---|---|
| Family viewer restrictions | `plus.family.child@lucatest.com` | View-only role |
| Admin dashboard | `admin@lucatest.com` | Admin but not super |
| System monitoring | `superadmin@lucatest.com` | Super admin only |
| Deployment rollback | `superadmin@lucatest.com` | Super admin only |
| MFA login flow | `mfa.enabled@lucatest.com` | Has MFA configured |
| Account lockout | `locked.account@lucatest.com` | Currently locked |
| New user onboarding | `free.newuser@lucatest.com` | No profile yet |

### 2.5 Environment Variables

```
# Test environment
NODE_ENV=test
DATABASE_URL=postgresql://test:test@localhost:5432/lucaagent_test
ADMIN_EMAIL_WHITELIST=superadmin@lucatest.com
SESSION_SECRET=test-session-secret-32-chars-min
```

### 2.6 Test Database Setup

```
# Reset test database
npm run db:test:reset

# Seed test users
npx tsx scripts/seed-test-users.ts
```

---

## 3. Authentication Test Suite

### 3.1 Registration Tests

| Test ID | Scenario | Steps | Expected Result |
|---|---|---|---|
| AUTH-REG-001 | Valid registration | POST `/api/auth/register` with valid data | 201 Created, user in DB |
| AUTH-REG-002 | Duplicate email | Register with existing email | 400 Error: "Email already exists" |
| AUTH-REG-003 | Weak password | Password < 8 chars, no special chars | 400 Error: Password validation |
| AUTH-REG-004 | Invalid email format | Register with "notanemail" | 400 Error: Invalid email |
| AUTH-REG-005 | Missing required fields | Omit name or password | 400 Error: Missing fields |

| | | | |
|---|---|---|---|
| AUTH-REG-006 | SQL injection attempt | Email: `'; DROP TABLE users;--` | 400 Error, no DB damage |

**Test Code:**

```
describe('Registration', () => {
  it('AUTH-REG-001: should register new user with valid data', async () => {
    const res = await request(app)
      .post('/api/auth/register')
      .send({
        email: 'newuser@test.com',
        password: 'SecurePass123!',
        name: 'New User',
      });

    expect(res.status).toBe(201);
    expect(res.body.user.email).toBe('newuser@test.com');
    expect(res.body.user.subscriptionTier).toBe('free');
    expect(res.body.user.password).toBeUndefined(); // Never expose
  });

  it('AUTH-REG-002: should reject duplicate email', async () => {
    // First registration
    await request(app).post('/api/auth/register').send({
      email: 'duplicate@test.com',
      password: 'SecurePass123!',
      name: 'First User',
    });

    // Duplicate attempt
    const res = await request(app).post('/api/auth/register').send({
      email: 'duplicate@test.com',
      password: 'SecurePass123!',
      name: 'Second User',
    });

    expect(res.status).toBe(400);
    expect(res.body.error).toContain('already exists');
  });
});
```

### 3.2 Login Tests

| Test ID | Scenario | Steps | Expected Result |
|---|---|---|---|
| AUTH-LOGIN-001 | Valid login | POST `/api/auth/login` with correct credentials | 200 OK, session cookie set |
| AUTH-LOGIN-002 | Wrong password | Login with incorrect password | 401 Unauthorized |

| | | | |
|---|---|---|---|
| AUTH-LOGIN-003 | Non-existent user | Login with unknown email | 401 Unauthorized |
| AUTH-LOGIN-004 | Account locked | 5 failed attempts, then correct password | 403 Account locked |
| AUTH-LOGIN-005 | Rate limiting | 11 login attempts in 15 min | 429 Too Many Requests |
| AUTH-LOGIN-006 | MFA required | Login when MFA enabled | 200 with `mfaRequired: true` |
| AUTH-LOGIN-007 | Valid MFA code | Submit correct TOTP after login | 200 OK, full session |
| AUTH-LOGIN-008 | Invalid MFA code | Submit wrong TOTP | 401 Invalid MFA code |
| AUTH-LOGIN-009 | MFA backup code | Use backup code instead of TOTP | 200 OK, backup code consumed |

**Test Code (using seeded accounts):**

```
const TEST_PASSWORD = 'TestPassword123!';

describe('Login', () => {
  it('AUTH-LOGIN-001: should login with valid credentials', async () => {
    const res = await request(app)
      .post('/api/auth/login')
      .send({
        email: 'free.home@lucatest.com',
        password: TEST_PASSWORD,
      });

    expect(res.status).toBe(200);
    expect(res.body.user).toBeDefined();
    expect(res.headers['set-cookie']).toBeDefined();
  });

  it('AUTH-LOGIN-004: should verify locked account', async () => {
    // Use pre-seeded locked account
    const res = await request(app).post('/api/auth/login').send({
      email: 'locked.account@lucatest.com',
      password: TEST_PASSWORD,
    });

    expect(res.status).toBe(403);
    expect(res.body.error).toContain('locked');
  });

  it('AUTH-LOGIN-006: should require MFA when enabled', async () => {
    const res = await request(app).post('/api/auth/login').send({
      email: 'mfa.enabled@lucatest.com',
```

```
      password: TEST_PASSWORD,
    });

    expect(res.status).toBe(200);
    expect(res.body.mfaRequired).toBe(true);
  });
});
```

### 3.3 Session Management Tests

| Test ID | Scenario | Expected Result |
|---|---|---|
| AUTH-SESSION-001 | Session persistence | Session survives server restart |
| AUTH-SESSION-002 | Session expiry | Session expires after configured timeout |
| AUTH-SESSION-003 | Logout | POST `/api/auth/logout` clears session |
| AUTH-SESSION-004 | Concurrent sessions | Multiple device logins allowed |
| AUTH-SESSION-005 | Session hijacking prevention | Session tied to user agent |

---

## 4. Role-Based Access Control Tests

### 4.1 Guest Access (No Authentication)

| Test ID | Endpoint | Expected Result |
|---|---|---|
| RBAC-GUEST-001 | GET `/api/health` | 200 OK |
| RBAC-GUEST-002 | GET `/api/feature-flags` | 200 OK |
| RBAC-GUEST-003 | GET `/api/chat` | 401 Unauthorized |
| RBAC-GUEST-004 | GET `/api/admin/dashboard` | 401 Unauthorized |
| RBAC-GUEST-005 | POST `/api/profiles` | 401 Unauthorized |

### 4.2 Regular User Access

| Test ID | Endpoint | Expected Result |
|---|---|---|
| RBAC-USER-001 | GET `/api/auth/me` | 200 OK with user data |
| RBAC-USER-002 | POST `/api/chat` | 200 OK (within quota) |
| RBAC-USER-003 | GET `/api/profiles` | 200 OK with own profiles |
| RBAC-USER-004 | GET `/api/admin/dashboard` | 403 Forbidden |
| RBAC-USER-005 | GET `/api/admin/users` | 403 Forbidden |
| RBAC-USER-006 | POST `/api/export` | 200 OK (tier-limited formats) |

**Test Code (using seeded test accounts):**

```
// Test account credentials - all use password: TestPassword123!
const TEST_ACCOUNTS = {
  freeUser: 'free.home@lucatest.com',
  plusUser: 'plus.business@lucatest.com',
  proUser: 'professional.cpa@lucatest.com',
  enterpriseUser: 'enterprise.owner@lucatest.com',
  admin: 'admin@lucatest.com',
  superAdmin: 'superadmin@lucatest.com',
};
const TEST_PASSWORD = 'TestPassword123!';

describe('Regular User RBAC', () => {
  let userCookie: string;

  beforeAll(async () => {
    // Login as free tier home user
    const res = await request(app).post('/api/auth/login').send({
      email: TEST_ACCOUNTS.freeUser,
      password: TEST_PASSWORD,
    });
    userCookie = res.headers['set-cookie'][0];
  });

  it('RBAC-USER-004: should deny admin dashboard access', async () => {
    const res = await request(app)
      .get('/api/admin/dashboard')
      .set('Cookie', userCookie);

    expect(res.status).toBe(403);
    expect(res.body.error).toContain('Admin access required');
  });

  it('RBAC-USER-002: should allow chat within quota', async () => {
    const res = await request(app)
      .post('/api/chat')
      .set('Cookie', userCookie)
      .send({ message: 'What is depreciation?' });

    expect(res.status).toBe(200);
    expect(res.body.response).toBeDefined();
  });
});
```

### 4.3 Admin Access

| Test ID | Endpoint | Expected Result |
|---|---|---|
| RBAC-ADMIN-001 | GET `/api/admin/dashboard` | 200 OK with KPIs |
| RBAC-ADMIN-002 | GET `/api/admin/users` | 200 OK with user list |

| | | |
|---|---|---|
| RBAC-ADMIN-003 | PATCH `/api/admin/users/:id` | 200 OK (can modify users) |
| RBAC-ADMIN-004 | GET `/api/admin/training-data` | 200 OK |
| RBAC-ADMIN-005 | POST `/api/admin/finetuning/trigger` | 200 OK |
| RBAC-ADMIN-006 | GET `/api/admin/system/monitoring` | 403 Forbidden (Super Admin only) |
| RBAC-ADMIN-007 | POST `/api/admin/system/deployments` | 403 Forbidden (Super Admin only) |

### 4.4 Super Admin Access

| Test ID | Endpoint | Expected Result |
|---|---|---|
| RBAC-SUPER-001 | GET `/api/admin/system/monitoring` | 200 OK with metrics |
| RBAC-SUPER-002 | GET `/api/admin/system/threats` | 200 OK with threat log |
| RBAC-SUPER-003 | POST `/api/admin/system/maintenance` | 200 OK |
| RBAC-SUPER-004 | POST `/api/admin/system/deployments/rollback` | 200 OK |
| RBAC-SUPER-005 | All admin endpoints | 200 OK (full access) |

**Test Code (using seeded accounts):**

```
const TEST_PASSWORD = 'TestPassword123!';

describe('Super Admin RBAC', () => {
  let superAdminCookie: string;

  beforeAll(async () => {
    // Login as super admin (must be in ADMIN_EMAIL_WHITELIST)
    const res = await request(app).post('/api/auth/login').send({
      email: 'superadmin@lucatest.com',
      password: TEST_PASSWORD,
    });
    superAdminCookie = res.headers['set-cookie'][0];
  });

  it('RBAC-SUPER-001: should access system monitoring', async () => {
    const res = await request(app)
      .get('/api/admin/system/monitoring')
      .set('Cookie', superAdminCookie);

    expect(res.status).toBe(200);
    expect(res.body.cpu).toBeDefined();
    expect(res.body.memory).toBeDefined();
  });

  it('RBAC-SUPER-003: should manage maintenance mode', async () => {
    const res = await request(app)
      .post('/api/admin/system/maintenance')
```

```
      .set('Cookie', superAdminCookie)
      .send({
        enabled: true,
        message: 'Scheduled maintenance',
        endsAt: new Date(Date.now() + 3600000).toISOString(),
      });

    expect(res.status).toBe(200);
  });
});
```

### 4.5 RBAC Matrix Verification

```
describe('Complete RBAC Matrix', () => {
  const endpoints = [
    { method: 'GET', path: '/api/auth/me', guest: 401, user: 200, admin: 200,
superAdmin: 200 },
    { method: 'POST', path: '/api/chat', guest: 401, user: 200, admin: 200,
superAdmin: 200 },
    { method: 'GET', path: '/api/admin/dashboard', guest: 401, user: 403, admin:
200, superAdmin: 200 },
    { method: 'GET', path: '/api/admin/users', guest: 401, user: 403, admin: 200,
superAdmin: 200 },
    { method: 'GET', path: '/api/admin/system/monitoring', guest: 401, user: 403,
admin: 403, superAdmin: 200 },
  ];

  endpoints.forEach(({ method, path, guest, user, admin, superAdmin }) => {
    it(`should enforce ${method} ${path} access correctly`, async () => {
      // Test each role...
    });
  });
});
```

---

## 5. Subscription Tier Tests

### 5.1 Free Tier Limits

| Test ID | Feature | Limit | Test Steps | Expected Result |
|---|---|---|---|---|
| TIER-FREE-001 | Chat queries | 500/month | Send 501st query | 429 Quota exceeded |
| TIER-FREE-002 | Document uploads | 10/month | Upload 11th document | 429 Quota exceeded |
| TIER-FREE-003 | Profiles | 1 | Create 2nd profile | 403 Upgrade required |

| | | | | |
|---|---|---|---|---|
| TIER-FREE-004 | Scenario Simulator | 0 | Access simulator | 403 Feature not available |
| TIER-FREE-005 | Export formats | TXT, CSV only | Export as PDF | 403 Upgrade required |

**Test Code (using seeded accounts):**

```
const TEST_PASSWORD = 'TestPassword123!';

describe('Free Tier Limits', () => {
  let freeCookie: string;
  let quotaExhaustedCookie: string;

  beforeAll(async () => {
    // Login as free home user
    const res = await request(app).post('/api/auth/login').send({
      email: 'free.home@lucatest.com',
      password: TEST_PASSWORD,
    });
    freeCookie = res.headers['set-cookie'][0];

    // Login as quota exhausted user for limit tests
    const quotaRes = await request(app).post('/api/auth/login').send({
      email: 'quota.exhausted@lucatest.com',
      password: TEST_PASSWORD,
    });
    quotaExhaustedCookie = quotaRes.headers['set-cookie'][0];
  });

  it('TIER-FREE-003: should block second profile creation', async () => {
    // First profile
    await request(app)
      .post('/api/profiles')
      .set('Cookie', freeCookie)
      .send({ name: 'First Profile', type: 'personal' });

    // Second profile attempt
    const res = await request(app)
      .post('/api/profiles')
      .set('Cookie', freeCookie)
      .send({ name: 'Second Profile', type: 'business' });

    expect(res.status).toBe(403);
    expect(res.body.error).toContain('upgrade');
  });

  it('TIER-FREE-005: should block PDF export', async () => {
    const res = await request(app)
      .post('/api/export')
      .set('Cookie', freeCookie)
```

```
      .send({
        conversationId: 'some-id',
        format: 'pdf',
      });

    expect(res.status).toBe(403);
    expect(res.body.error).toContain('PDF export requires Plus tier');
  });
});
```

### 5.2 Plus Tier Features

| Test ID | Feature | Test Steps | Expected Result |
|---|---|---|---|
| TIER-PLUS-001 | 3000 queries/month | Send queries within limit | 200 OK |
| TIER-PLUS-002 | Unlimited documents | Upload 100+ documents | 200 OK |
| TIER-PLUS-003 | 5 profiles | Create 5 profiles | 200 OK |
| TIER-PLUS-004 | 6th profile | Create 6th profile | 403 Upgrade required |
| TIER-PLUS-005 | Scenario Simulator | 10/month | 200 OK |
| TIER-PLUS-006 | All export formats | Export as PDF, DOCX | 200 OK |
| TIER-PLUS-007 | Forensic Intelligence | Access forensic | 403 Not available |

### 5.3 Professional Tier Features

| Test ID | Feature | Test Steps | Expected Result |
|---|---|---|---|
| TIER-PRO-001 | Unlimited queries | 10,000+ queries | 200 OK |
| TIER-PRO-002 | Unlimited profiles | 100+ profiles | 200 OK |
| TIER-PRO-003 | API access | Use API key | 200 OK |
| TIER-PRO-004 | Forensic Intelligence | Access forensic tools | 200 OK |
| TIER-PRO-005 | White-label exports | Custom branding | 200 OK |
| TIER-PRO-006 | SSO/SAML | Attempt SSO login | 403 Enterprise only |

### 5.4 Enterprise Tier Features

| Test ID | Feature | Test Steps | Expected Result |
|---|---|---|---|
| TIER-ENT-001 | All professional features | All tests from PRO | 200 OK |
| TIER-ENT-002 | SSO/SAML | Configure SSO | 200 OK |
| TIER-ENT-003 | Multi-user (6+) | Add 10 team members | 200 OK |
| TIER-ENT-004 | Custom AI training | Submit training data | 200 OK |

| | | | |
|---|---|---|---|
| TIER-ENT-005 | SLA monitoring | Check SLA dashboard | 200 OK with 99.9% target |

### 5.5 Quota Reset Tests

| Test ID | Scenario | Expected Result |
|---|---|---|
| TIER-RESET-001 | Monthly quota reset | Quota resets on 1st of month |
| TIER-RESET-002 | Mid-month upgrade | Immediate quota increase |
| TIER-RESET-003 | Mid-month downgrade | Retains current month quota |

---

## 6. Feature Tests by Role

### 6.1 Chat Features

| Test ID | User Type | Feature | Expected Result |
|---|---|---|---|
| CHAT-001 | Free | Basic tax question | Response generated |
| CHAT-002 | Free | Complex multi-part question | Simplified response |
| CHAT-003 | Plus | Deep research mode | Comprehensive analysis |
| CHAT-004 | Pro | Forensic analysis query | Fraud indicators provided |
| CHAT-005 | All | Chat history persistence | Previous chats accessible |
| CHAT-006 | All | Chat export | Export in allowed formats |
| CHAT-007 | All | File attachment in chat | Document analyzed in context |

**Test Code:**

```
describe('Chat Features', () => {
  it('CHAT-003: Plus user can use deep research mode', async () => {
    const res = await request(app)
      .post('/api/chat')
      .set('Cookie', plusCookie)
      .send({
        message: 'Analyze transfer pricing implications for cross-border
transactions',
        mode: 'deep-research',
      });

    expect(res.status).toBe(200);
    expect(res.body.mode).toBe('deep-research');
    expect(res.body.response.length).toBeGreaterThan(1000); // Comprehensive
  });

  it('CHAT-004: Professional user can access forensic analysis', async () => {
    const res = await request(app)
      .post('/api/chat')
```

```
      .set('Cookie', proCookie)
      .send({
        message: 'Analyze these transactions for fraud indicators',
        mode: 'forensic',
        attachments: [testTransactionFile],
      });

    expect(res.status).toBe(200);
    expect(res.body.forensicFlags).toBeDefined();
  });
});
```

### 6.2 Profile Management

| Test ID | User Type | Action | Expected Result |
|---|---|---|---|
| PROF-001 | All | Create personal profile | Profile created |
| PROF-002 | Plus+ | Create business profile | Profile created |
| PROF-003 | Plus+ | Create family profile | Profile created |
| PROF-004 | Owner | Delete own profile | Profile deleted |
| PROF-005 | Owner | Invite member | Invitation sent |
| PROF-006 | Admin (member) | Edit profile settings | Settings updated |
| PROF-007 | Member | View profile data | Data visible |
| PROF-008 | Member | Invite new member | 403 Forbidden |
| PROF-009 | Viewer | Edit profile data | 403 Forbidden |
| PROF-010 | Non-member | Access profile | 403 Forbidden |

### 6.3 Document Management

| Test ID | User Type | Action | Expected Result |
|---|---|---|---|
| DOC-001 | All | Upload PDF | Document analyzed |
| DOC-002 | All | Upload image | OCR processed |
| DOC-003 | All | Upload Excel | Data extracted |
| DOC-004 | Free | Exceed document limit | 429 Quota exceeded |
| DOC-005 | Plus+ | Unlimited uploads | All uploads succeed |
| DOC-006 | All | Download own document | 200 OK |
| DOC-007 | All | Download other's document | 403 Forbidden |
| DOC-008 | All | Virus in upload | 400 Malware detected |

### 6.4 Excel Generation

| Test ID | User Type | Template | Expected Result |
|---|---|---|---|
| EXCEL-001 | All | NPV calculation | Excel with formulas |
| EXCEL-002 | All | Depreciation schedule | Excel with formulas |
| EXCEL-003 | Plus+ | Scenario analysis | Multi-scenario workbook |
| EXCEL-004 | Pro | Custom template | Branded Excel output |

### 6.5 Admin Features

| Test ID | Admin Type | Feature | Expected Result |
|---|---|---|---|
| ADMIN-001 | Admin | View dashboard | KPIs displayed |
| ADMIN-002 | Admin | Search users | User list returned |
| ADMIN-003 | Admin | Edit user tier | Tier updated |
| ADMIN-004 | Admin | Create coupon | Coupon created |
| ADMIN-005 | Admin | View training data | Data displayed |
| ADMIN-006 | Admin | Approve training example | Example approved |
| ADMIN-007 | Admin | Trigger fine-tuning | Job queued |
| ADMIN-008 | Super Admin | View system metrics | Metrics displayed |
| ADMIN-009 | Super Admin | Enable maintenance | System in maintenance |
| ADMIN-010 | Super Admin | Rollback deployment | Deployment reverted |

---

## 7. Profile & Member Tests

### 7.1 Profile Creation Workflows

```
describe('Profile Creation', () => {
  it('PROF-CREATE-001: Personal profile auto-assigns owner role', async () => {
    const res = await request(app)
      .post('/api/profiles')
      .set('Cookie', userCookie)
      .send({
        name: 'My Taxes',
        type: 'personal',
      });

    expect(res.status).toBe(201);
    expect(res.body.members[0].role).toBe('owner');
    expect(res.body.members[0].userId).toBe(currentUserId);
  });
```

```
  it('PROF-CREATE-002: Business profile requires tier check', async () => {
    // Free user
    const freeRes = await request(app)
      .post('/api/profiles')
      .set('Cookie', freeCookie)
      .send({ name: 'My Business', type: 'business' });

    expect(freeRes.status).toBe(403);

    // Plus user
    const plusRes = await request(app)
      .post('/api/profiles')
      .set('Cookie', plusCookie)
      .send({ name: 'My Business', type: 'business' });

    expect(plusRes.status).toBe(201);
  });
});
```

### 7.2 Member Invitation Flow

| Step | Action | Actor | Expected Result |
|---|---|---|---|
| 1 | Invite member by email | Owner | Invitation created |
| 2 | Non-existent email | Owner | User created + invited |
| 3 | Accept invitation | Invitee | Member added to profile |
| 4 | Reject invitation | Invitee | Invitation deleted |
| 5 | Invitation expires | System | Auto-deleted after 7 days |

```
describe('Member Invitation Flow', () => {
  it('should complete full invitation workflow', async () => {
    // Step 1: Owner invites member
    const inviteRes = await request(app)
      .post('/api/profiles/123/members/invite')
      .set('Cookie', ownerCookie)
      .send({
        email: 'newmember@test.com',
        role: 'member',
      });

    expect(inviteRes.status).toBe(201);
    const invitationToken = inviteRes.body.token;

    // Step 2: Invitee accepts
    const acceptRes = await request(app)
      .post('/api/invitations/accept')
      .set('Cookie', newMemberCookie)
```

```
      .send({ token: invitationToken });

    expect(acceptRes.status).toBe(200);

    // Verify membership
    const profileRes = await request(app)
      .get('/api/profiles/123')
      .set('Cookie', newMemberCookie);

    expect(profileRes.body.members).toContainEqual(
      expect.objectContaining({
        email: 'newmember@test.com',
        role: 'member',
      })
    );
  });
});
```

### 7.3 Member Role Permissions Matrix

| Action | Owner | Admin | Member | Viewer |
|---|---|---|---|---|
| View profile | ✅ | ✅ | ✅ | ✅ |
| Edit profile settings | ✅ | ✅ | ❌ | ❌ |
| Add data/documents | ✅ | ✅ | ✅ | ❌ |
| Invite members | ✅ | ✅ | ❌ | ❌ |
| Change member roles | ✅ | ❌ | ❌ | ❌ |
| Remove members | ✅ | ✅ (non-admins) | ❌ | ❌ |
| Delete profile | ✅ | ❌ | ❌ | ❌ |
| Transfer ownership | ✅ | ❌ | ❌ | ❌ |

```
describe('Member Role Permissions', () => {
  const actions = [
    { action: 'edit-settings', owner: 200, admin: 200, member: 403, viewer: 403 },
    { action: 'add-document', owner: 200, admin: 200, member: 200, viewer: 403 },
    { action: 'invite-member', owner: 200, admin: 200, member: 403, viewer: 403 },
    { action: 'delete-profile', owner: 200, admin: 403, member: 403, viewer: 403 },
  ];

  actions.forEach(({ action, ...expected }) => {
    it(`should enforce ${action} permissions`, async () => {
      // Test each role against expected status code
    });
  });
});
```

---

## 8. Cross-Role Interaction Tests

### 8.1 Admin Modifying User

| Test ID | Scenario | Expected Result |
|---|---|---|
| CROSS-001 | Admin upgrades user tier | User gains new features immediately |
| CROSS-002 | Admin disables user account | User's active sessions terminated |
| CROSS-003 | Admin resets user password | User receives reset email |
| CROSS-004 | Admin views user conversations | Conversations visible (audit logged) |
| CROSS-005 | Admin cannot modify Super Admin | 403 Forbidden |

```
describe('Admin-User Interactions', () => {
  it('CROSS-001: Admin upgrade reflects immediately', async () => {
    // Admin upgrades free user to plus
    await request(app)
      .patch(`/api/admin/users/${freeUserId}`)
      .set('Cookie', adminCookie)
      .send({ subscriptionTier: 'plus' });

    // Free user now has plus features
    const res = await request(app)
      .post('/api/profiles')
      .set('Cookie', freeCookie) // Same cookie, but tier changed
      .send({ name: 'New Business', type: 'business' });

    expect(res.status).toBe(201); // Was 403 before upgrade
  });

  it('CROSS-005: Admin cannot modify Super Admin', async () => {
    const res = await request(app)
      .patch(`/api/admin/users/${superAdminId}`)
      .set('Cookie', adminCookie)
      .send({ subscriptionTier: 'free' });

    expect(res.status).toBe(403);
    expect(res.body.error).toContain('Cannot modify super admin');
  });
});
```

### 8.2 Profile Member Interactions

| Test ID | Scenario | Expected Result |
|---|---|---|
| CROSS-010 | Owner adds viewer | Viewer sees profile in their list |
| CROSS-011 | Owner removes member | Member loses access immediately |

| | | |
|---|---|---|
| CROSS-012 | Owner transfers to admin | Old owner becomes admin, new owner has full control |
| CROSS-013 | Admin invites member | Member receives invitation |
| CROSS-014 | Member views owner's documents | Documents visible |

### 8.3 Concurrent Access

| Test ID | Scenario | Expected Result |
|---|---|---|
| CROSS-020 | Two users edit same profile | Last write wins (conflict resolution) |
| CROSS-021 | Admin deletes user mid-session | User's requests fail gracefully |
| CROSS-022 | Tier downgrade during export | Export completes, then limits apply |

---

## 9. Security Tests

### 9.1 Authentication Security

| Test ID | Attack Vector | Expected Defense |
|---|---|---|
| SEC-AUTH-001 | Brute force login | Account lockout after 5 attempts |
| SEC-AUTH-002 | Session fixation | New session ID on login |
| SEC-AUTH-003 | Session hijacking | Session bound to user agent |
| SEC-AUTH-004 | CSRF on state-changing | CSRF token required |
| SEC-AUTH-005 | XSS in user input | Input sanitized |

### 9.2 Authorization Security

| Test ID | Attack Vector | Expected Defense |
|---|---|---|
| SEC-AUTHZ-001 | IDOR (access other user's data) | 403 Forbidden |
| SEC-AUTHZ-002 | Privilege escalation | Role cannot be self-modified |
| SEC-AUTHZ-003 | Forced browsing to admin | 403 without admin role |
| SEC-AUTHZ-004 | API key without permission | 403 for disallowed endpoints |

```
describe('IDOR Prevention', () => {
  it('SEC-AUTHZ-001: cannot access other user profile', async () => {
    // User A creates profile
    const profile = await request(app)
      .post('/api/profiles')
      .set('Cookie', userACookie)
```

```
      .send({ name: 'Private Profile', type: 'personal' });

    // User B tries to access
    const res = await request(app)
      .get(`/api/profiles/${profile.body.id}`)
      .set('Cookie', userBCookie);

    expect(res.status).toBe(403);
  });

  it('SEC-AUTHZ-002: cannot escalate own role', async () => {
    const res = await request(app)
      .patch('/api/users/me')
      .set('Cookie', userCookie)
      .send({ isAdmin: true });

    // isAdmin should be ignored or rejected
    expect(res.body.isAdmin).toBe(false);
  });
});
```

### 9.3 File Upload Security

| Test ID | Attack Vector | Expected Defense |
|---|---|---|
| SEC-FILE-001 | Malware upload | Virus scan blocks |
| SEC-FILE-002 | Path traversal in filename | Filename sanitized |
| SEC-FILE-003 | Oversized file | Size limit enforced |
| SEC-FILE-004 | Wrong MIME type | Content-type validated |
| SEC-FILE-005 | Executable disguised as PDF | Magic bytes checked |

### 9.4 Rate Limiting

| Test ID | Endpoint | Limit | Window |
|---|---|---|---|
| SEC-RATE-001 | `/api/auth/*` | 10 requests | 15 minutes |
| SEC-RATE-002 | `/api/chat/*` | 20 requests | 1 minute |
| SEC-RATE-003 | File uploads | 20 requests | 15 minutes |
| SEC-RATE-004 | OAuth flows | 5 requests | 15 minutes |

---

## 10. Edge Cases & Error Handling

### 10.1 Boundary Conditions

| Test ID | Scenario | Expected Result |
|---|---|---|

| | | |
|---|---|---|
| EDGE-001 | Query at exact quota limit | 200 OK |
| EDGE-002 | Query 1 over quota | 429 with clear message |
| EDGE-003 | 0-length message | 400 Invalid input |
| EDGE-004 | Max-length message (10MB) | 413 Payload too large |
| EDGE-005 | Unicode/emoji in all fields | Handled correctly |
| EDGE-006 | Concurrent rapid requests | Rate limited, no crashes |

### 10.2 Error Response Consistency

```
describe('Error Response Format', () => {
  it('should return consistent error structure', async () => {
    const res = await request(app)
      .post('/api/chat')
      .send({}); // Missing auth

    expect(res.status).toBe(401);
    expect(res.body).toMatchObject({
      error: expect.any(String),
      code: expect.any(String), // e.g., "UNAUTHORIZED"
      timestamp: expect.any(String),
    });
  });
});
```

### 10.3 Graceful Degradation

| Test ID | Failure | Expected Behavior |
|---|---|---|
| GRACE-001 | AI provider down | Fallback to backup provider |
| GRACE-002 | Database slow | Request times out gracefully |
| GRACE-003 | Redis unavailable | Falls back to in-memory cache |
| GRACE-004 | Email service down | Queues email for retry |

---

## 11. Performance Tests

### 11.1 Response Time Benchmarks

| Endpoint | Target | Max Acceptable |
|---|---|---|
| GET `/api/auth/me` | < 50ms | 200ms |
| POST `/api/chat` | < 3s | 10s |
| GET `/api/profiles` | < 100ms | 500ms |

| | | |
|---|---|---|
| POST `/api/export` (PDF) | < 5s | 30s |
| GET `/api/admin/dashboard` | < 200ms | 1s |

### 11.2 Load Testing Scenarios

```
describe('Load Tests', () => {
  it('should handle 100 concurrent users', async () => {
    const users = Array.from({ length: 100 }, (_, i) =>
      request(app)
        .post('/api/chat')
        .set('Cookie', cookies[i])
        .send({ message: 'Hello' })
    );

    const results = await Promise.all(users);
    const successRate = results.filter(r => r.status === 200).length / 100;

    expect(successRate).toBeGreaterThan(0.95);
  });

  it('should maintain response times under load', async () => {
    const start = Date.now();
    await request(app).get('/api/auth/me').set('Cookie', cookie);
    const duration = Date.now() - start;

    expect(duration).toBeLessThan(500);
  });
});
```

---

## 12. Test Data Management

### 12.1 Test Data Seeding

```
// scripts/seed-test-data.ts
export async function seedTestData() {
  // Create test users
  for (const [key, user] of Object.entries(TEST_USERS)) {
    await db.insert(users).values({
      ...user,
      password: await bcrypt.hash(user.password, 10),
    });
  }

  // Create test profiles
  await db.insert(profiles).values([
    { name: 'Test Personal', type: 'personal', userId: freeUserId },
    { name: 'Test Business', type: 'business', userId: corporateUserId },
    { name: 'Test Family', type: 'family', userId: plusUserId },
```

```
  ]);

  // Create test conversations
  await db.insert(conversations).values([
    { userId: freeUserId, title: 'Tax Question', profileId: profile1Id },
  ]);
}
```

### 12.2 Test Data Cleanup

```
// After each test suite
afterAll(async () => {
  // Clean in reverse dependency order
  await db.delete(messages).where(like(messages.content, 'TEST%'));
  await db.delete(conversations).where(like(conversations.title, 'Test%'));
  await db.delete(profileMembers).where(sql`profile_id IN (SELECT id FROM profiles
WHERE name LIKE 'Test%')`);
  await db.delete(profiles).where(like(profiles.name, 'Test%'));
  await db.delete(users).where(like(users.email, '%lucaagent-test.com'));
});
```

### 12.3 Test Fixtures

```
// fixtures/users.fixture.ts
export const createTestUser = async (overrides = {}) => {
  const defaults = {
    email: `test-${Date.now()}@lucaagent-test.com`,
    password: 'TestPassword123!',
    name: 'Test User',
    subscriptionTier: 'free',
    isAdmin: false,
  };

  const user = { ...defaults, ...overrides };
  const created = await db.insert(users).values({
    ...user,
    password: await bcrypt.hash(user.password, 10),
  }).returning();

  return { ...created[0], plainPassword: user.password };
};

export const loginAs = async (user: { email: string; password: string }) => {
  const res = await request(app)
    .post('/api/auth/login')
    .send({ email: user.email, password: user.password });

  return res.headers['set-cookie'][0];
};
```

---

## Appendix A: Complete Test Matrix

### A.1 Role × Feature Matrix

| Feature | Guest | Free | Plus | Pro | Enterprise | Admin | Super Admin |
|---|---|---|---|---|---|---|---|
| View landing page | ✅ | ✅ | ✅ | ✅ | ✅ | ✅ | ✅ |
| Register | ✅ | N/A | N/A | N/A | N/A | N/A | N/A |
| Login | ✅ | N/A | N/A | N/A | N/A | N/A | N/A |
| Chat | ❌ | 500/mo | 3K/mo | ∞ | ∞ | ∞ | ∞ |
| Documents | ❌ | 10/mo | ∞ | ∞ | ∞ | ∞ | ∞ |
| Profiles | ❌ | 1 | 5 | ∞ | ∞ | ∞ | ∞ |
| Scenario Sim | ❌ | ❌ | 10/mo | ∞ | ∞ | ∞ | ∞ |
| Forensic | ❌ | ❌ | ❌ | ✅ | ✅ | ✅ | ✅ |
| API Access | ❌ | ❌ | ❌ | ✅ | ✅ | ✅ | ✅ |
| SSO/SAML | ❌ | ❌ | ❌ | ❌ | ✅ | ✅ | ✅ |
| Admin Dashboard | ❌ | ❌ | ❌ | ❌ | ❌ | ✅ | ✅ |
| User Management | ❌ | ❌ | ❌ | ❌ | ❌ | ✅ | ✅ |
| System Monitoring | ❌ | ❌ | ❌ | ❌ | ❌ | ❌ | ✅ |
| Deployments | ❌ | ❌ | ❌ | ❌ | ❌ | ❌ | ✅ |

### A.2 API Endpoint × Role Matrix

See [API_ENDPOINTS_REFERENCE.md](API_ENDPOINTS_REFERENCE.md) for complete endpoint documentation.

---

## Appendix B: Running Tests

### B.1 Commands

```
# Run all E2E tests
npm run test:e2e

# Run specific test suite
npm run test:e2e -- --grep "Authentication"

# Run with coverage
npm run test:e2e -- --coverage
```

```
# Run in watch mode
npm run test:e2e -- --watch

# Run specific role tests
npm run test:e2e -- --grep "RBAC-ADMIN"
```

### B.2 CI/CD Integration

```yaml
# .github/workflows/e2e-tests.yml
name: E2E Tests
on: [push, pull_request]

jobs:
  e2e:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_DB: lucaagent_test
          POSTGRES_USER: test
          POSTGRES_PASSWORD: test
        ports:
          - 5432:5432
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with:
          node-version: '20'
      - run: npm ci
      - run: npm run db:test:setup
      - run: npm run test:e2e
```

---

## Appendix C: Test IDs Reference

| Prefix | Category | Count |
|---|---|---|
| AUTH-* | Authentication | 20+ |
| RBAC-* | Role-Based Access | 30+ |
| TIER-* | Subscription Tiers | 25+ |
| CHAT-* | Chat Features | 10+ |
| PROF-* | Profile Management | 15+ |
| DOC-* | Documents | 10+ |
| ADMIN-* | Admin Features | 15+ |

| | | |
|---|---|---|
| CROSS-* | Cross-Role Interactions | 25+ |
| SEC-* | Security | 20+ |
| EDGE-* | Edge Cases | 10+ |
| GRACE-* | Graceful Degradation | 5+ |
| PERF-* | Performance | 10+ |

**Total Test Cases: 200+**

---

*This guide should be updated whenever new roles, features, or endpoints are added to LucaAgent.*